מקליד: עידן אריה
מתרגל: דוד יצחקי
תאריך: 11-06-2026

בינה מלאכותית
89-5570-10

למידת חיזוקים בLarge State Spaces

עד עכשיו עבדנו עם tabular RL, שבו יכולנו לשמור ערך לכל מצב - או ערך לכל קומבינציה של מצב ופעולה. זה היה אפשרי כי מספר המצבים $|\mathcal{S}|$ ומספר הפעולות $|\mathcal{A}|$ היו קטנים, ויכולנו לבקר באותו מצב הרבה פעמים ולשפר את הערך בטבלה.

אבל מה אם, למשל, המצב שלנו זה תמונה? רוחב W, גובה H, ומספר ערוצים (למשל צבעים) K - אז עבור תמונת RGB בגודל 100×100 יש $100 \cdot 100 \cdot 3 = 30,000$ ערכים - ואם כל אחד מהם יכול להיות בין 0 ל255 אז יש לנו $|\mathcal{S}| = 265^{30,000}$ שזה יותר מדי גדול. ואם נותנים ערכים לפעולות אז זה עוד יותר גדול - $|\mathcal{S}| \cdot |\mathcal{A}| = 4 \cdot 256^{30,000}$.

נרצה ללמוד בצורה שמצבים שהם מאוד דומים יקבלו את אותם ערכים, וככה הם ילמדו דברים דומים.

אנחנו רוצים הכללה. רוצים לחלץ פיצ'רים שיכלילו.

תזכורת: יש שתי שאלות שונות:

1. מה הexpected return של פוליסה π ממצב s?

$$V^{\pi}(s) = \mathbb{E}_{\pi}[G_t | S_t = s]$$

2. מה הexpected return אם מבצעים פעולה a במצב s, ואז ממשיכים עם פוליסה π?

$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}[G_t | S_t = s, A_t = a]$$

במקום טבלה, נשתמש בפונקציה עם פרמטרים θ כדי לשערך את הערכים האלו. כלומר $V_{\theta}(s)$ שיתן ערך משוערך.

מספר הפרמטרים צריך להיות יותר קטן ממספר המצבים. נשים לב שבמקרה של מספר הפרמטרים מסתכלים על מספר המשתנים ולא על מספר הקומבינציות שלהם, בעוד שבמקרה של $|\mathcal{S}|$ מסתכלים על מספר המצבים.

אפשר, למשל, לחלץ פיצ'רים שיכולים להיקבע ע"י בני אדם, או להילמד ע"י מודל:

$$\phi(s) = (f_1(s), f_2(s), \ldots, f_n(s))$$

ואז אפשר לעשות התמרה לינארית:

$$V_{\theta}(s) = \theta^T \phi(s)$$

נשים לב שפיצ'רים לינארים לא תמיד מספיקה, ולכן אפשר להוסיף פיצ'רים חדשים.

איך בוחרים את הפיצ'רים? ואת הפרמטרים? ידנית? לא טוב. נרצה להשתמש בלמידת מכונה בשביל זה.

נתחיל בבחירת d פיצ'רים ידנית, ונלמד את θ. איך לומדים?

בלמידה מפוקחת - מקבלים דוגמאות עם ערכים:

$$(s_1, v(s_1)), (s_2, v(s_2)), \ldots, (s_m, v(s_m))$$

אנחנו יכולים לייצר את הערכים האלו בעצמנו באמצעות סימולציה.
מחשבים את השגיאה:

$$E(s) = \frac{1}{2}\left(V_\theta(s) - v(s)\right)^2$$

ומבצעים gradient descent.
אנחנו לא יודעים את הערך האמיתי $v(s)$, אבל אם יש לנו ערך במרחק צעד אז TD Target נותן:

$$v(s) = r + \gamma V_\theta(s')$$

ואז אפשר לעדכן:

$$\theta_i \leftarrow \theta_i + \alpha\left(r + \gamma V_\theta(s') - V_\theta(s)\right) f_i(s)$$

בשביל למצוא פעולה שממקסמת את הערך מחפשים

$$\pi(s) = \arg\max_a Q_\theta(s,a)$$

והעדכון הוא

$$\theta_i \leftarrow \theta_i + \alpha\left(r + \gamma \max_{a'} Q_\theta(s',a') - Q_\theta(s,a)\right) f_i(s,a)$$

זה לא תמיד מתכנס - אבל אין מה לעשות, מרחב המצבים יותר מדי גדול.

מקליד: עידן אריה
מתרגל: דוד יצחקי
תאריך: 11-06-2026

בינה מלאכותית
89-5570-10

Deep Q-Networks - DQN

בlinear approximation היינו צריכים לבחור בעצמנו את הפיצ'רים. ברשתות עמוקות יש hidden layers שלומדות את הפיצ'רים.

אפשר להסתכל על משחקי אטארי בתור MDP. הסוכן רואה את המסך, בוחר פעולה, המשחק מעדכן את המשחק, והסוכן רואה את הreward לפי הניקוד.

לפני DQN, לכל משחק הגדירו בדיוק פרמטרים ספציפיים ופיצ'רים ספציפיים ולא הייתה הכללה.

הערה: אמנם נצטרך לאמן את המודל על כל משחק בנפרד - אבל נרצה להשתמש באותה ארכיטקטורה ואותם היפטרפרמטרים.

DQN פותר את זה - משתמש באותה ארכיטקטורה, אותו אלגוריתם למידה, ואותם היפרפרמטרים. רק האימון השתנה בין משחק למשחק.

הם השתמשו ברשת ניורונים בשביל לחשב את ערך הQ:

$$Q(s, a; \theta) \approx Q^*(s, a)$$

ולפי זה בחרו בכל צעד את הפעולה הכי טובה:

$$\pi(s) = \arg\max_a Q(s, a; \theta)$$

איך הכלילו את הstateים? הDQN קיבל תמונה שמורכבת מפיקסלים. התמונה מאוד גדולה - ולכן קודם כל:

- חתכו את החלק שמציג את הניקוד.
- שינו את הגודל של התמונה ל84×84 (שיהיה ריבועי וקטן)
- הפכו את התמונות לשחור־לבן (כדי לא ללמוד את הצבעים)

פריים יחיד לא מספיק - כי הוא לא מראה את התנועה (הכיוון בו דברים זזים). לכן לקחו כמה פריימים ביחד:

$$\phi_t = (x_{t-3}, x_{t-2}, x_{t-1}, x_t)$$

את זה הכניסו לDQN שמתחיל בCNNים[1]. ומוציא ערכים לכל פעולה.

העדכון מתבצע באמצעות Experience Replay Buffer - זיכרון שהסוכן יכול לשמור. כל מיקום בbuffer שומר transition:

$$(\phi_t, a_t, r_t, \phi_{t+1})$$

ϕ_t המצב הנוכחי (4 פריימים)

[1] Convolutional Neural Network

מקליד: עידן אריה
מתרגל: דוד יצחקי
תאריך: 11-06-2026

בינה מלאכותית
89-5570-10

---

a_t הפעולה שבחרנו

r_t התגמול שקיבלנו

ϕ_{t+1} המצב שהגענו אליו

הDQN במאמר מ2013 שמר buffer בגודל מיליון. כשהוא התמלא, זרקו את הערכים הישנים. הbuffer הזה בעצם ייצג את הtarget policy.

בעיות:

1. אם מאמנים רק על transitionים חדשים, הdata מאוד correlated. צריך הרבה זמן במשחק עד שיש שינוי.
   פותרים את זה עם mini-batch, שדוגמים מתוך הbuffer ומאמנים עליו.
   עד שאין מספיק נתונים בbuffer, אי אפשר להתאמן והאלגוריתם מבצע פעולות באקראי.
2. תוך כדי תנועה, ערכי Q משתנים - ואז גם ההתפלגות משתנה.
   הreplay buffer מתקן את זה, כי הוא מאפשר לדגום מהעבר.
3. יכול להיות transition שמלמד הרבה. היינו רוצים איכשהו לדגום את הדבר הזה הרבה. הreplay buffer הtransition הזה נשמר ואפשר ללמוד ממנו הרבה פעמים.

איך עושים את זה בפועל? צריך להגדיר target:

$$y_j = \begin{cases} r_j & \phi_{j+1} \text{ is terminal} \\ r_j + \gamma \max_{a'} Q\left(\phi_{j+1}, a'; \theta\right) & \phi_{j+1} \text{ is not terminal} \end{cases}$$

מה זה terminal? כשנפסלנו במשחק.
רוצים להקטין את $\left(y_j - Q\left(\phi_j, a_j; \theta\right)\right)^2$, ועושים את זה באמצעות gradient descent.